Звіт: дослідження std::inclusive_scan (варіант 12)

Умови. g++, C++20, процесор із 12 потоками (hardware_concurrency = 12). Рівні оптимізації: -O0 та -O3. Послідовності: N = 10^5, 10^6, 10^7 значень типу long long з [1; 10^9+6] (mt19937_64), згенеровані заздалегідь у файли й однакові для всіх експериментів. Операції: Add (швидка) та MulMod, (a·b) mod (10^9+7) (важка). Час — мінімум із 5 запусків, мс.

1–2. Бібліотечний алгоритм: без політики та з політиками

Операція	N	O0 без	O0 seq	O0 par	O0 par_us	O3 без	O3 seq	O3 par	O3 par_us
Add	10^5	0.291	0.294	0.293	0.280	0.035	0.035	0.035	0.035
Add	10^6	3.14	3.36	3.18	3.24	0.80	0.63	0.57	0.60
Add	10^7	31.3	32.6	32.9	32.7	9.64	9.55	9.56	9.66
MulMod	10^5	0.425	0.317	0.315	0.315	0.327	0.327	0.327	0.327
MulMod	10^6	3.11	3.72	3.58	3.60	3.03	3.12	3.09	3.07
MulMod	10^7	31.7	34.0	37.2	36.6	31.4	31.5	31.5	31.4

Висновки. Час росте лінійно з N (×10 при ×10 за N). Політики seq/par/par_unseq (у таблиці par_us = par_unseq) не дають прискорення: різниця в межах похибки (імовірно, бібліотека виконує їх послідовно, бо збірка без TBB-бекенду). Оптимізація -O3 прискорює Add у 3.3× (N=10^7) і до 8× (N=10^5, дані у кеші), бо при -O0 час визначають виклики неінлайнених функцій та ітераторів (Add і MulMod при -O0 однакові, ≈31 мс). MulMod від оптимізації майже не виграє (31.7→31.4 мс): час визначає ланцюг залежних множень і ділень за модулем.

3. Власний паралельний алгоритм (K потоків)

Вибірка ділиться на K частин; кожна обробляється окремим потоком (std::inclusive_scan); підсумки частин зводяться послідовним inclusive_scan; потім до частин 2..K додається префікс (ще один прохід даних у K потоках). Коректність перевірено порівнянням із бібліотечним результатом (OK).

Час (мс) залежно від K, N=10^7

K	Add O0	Add O3	MulMod O0	MulMod O3
1	32.09	10.12	33.07	31.39
2	32.92	13.04	34.77	20.75
4	18.71	15.32	19.81	15.48
8	17.98	15.83	19.57	16.31
12	16.11	15.92	16.84	16.54
16	16.88	16.08	17.52	16.21
64	16.97	16.73	17.39	16.78
256	23.09	20.46	23.64	21.13
1024	76.67	74.77	77.33	75.19

Найкраще K (час, мс)

Режим	N=10^5	N=10^6	N=10^7
Add, O0	1 (0.54)	16 (1.88)	12 (16.1)
MulMod, O0	1 (0.51)	12 (2.04)	12 (16.8)
Add, O3	1 (0.35)	1 (1.07)	1 (10.1)
MulMod, O3	1 (0.53)	6 (1.31)	4 (15.5)

Повні таблиці для всіх K і N — у файлах out_O0.txt та out_O3.txt.

Яке K найкраще і як воно відноситься до кількості потоків (12). Для великих N при -O0 найкраще K = 12 (16 для Add, N=10^6), тобто K ≈ кількості потоків процесора (K/12 ≈ 1); виграш ≈ 2× (32→16 мс). При -O3 для важкої операції найкраще K = 4–6 (K/12 ≈ 0.3–0.5), виграш також ≈ 2× (31.4→15.5 мс). Для швидкої операції Add при -O3 і для N=10^5 найкраще K = 1: послідовний прохід настільки швидкий (≈1 нс/елемент, обмежений пам’яттю), що витрати на створення потоків і другий прохід не окупаються; власний алгоритм при K=1 (10.1 мс) дорівнює бібліотечному (9.6 мс).

Закон зростання часу зі збільшенням K. Після оптимуму (плато ≈16 мс для N=10^7, яке однакове для обох операцій і рівнів оптимізації — тобто його визначають пропускна здатність пам’яті та накладні витрати, а не обчислення) час зростає лінійно: $T(K) \approx T_0 + c \cdot K$, c ≈ 0.07 мс на потік (≈35 мкс на створення й завершення одного потоку в кожній із двох паралельних фаз). Це видно по подвоєнню K: для N=10^5 час при K = 128, 256, 512, 1024 становить 8.4, 16.9, 34.2, 71.9 мс — кожне подвоєння K подвоює час, незалежно від N.

Загальні висновки

1) Бібліотечні політики виконання в цьому середовищі прискорення не дали. 2) -O3 суттєво прискорює лише легку операцію; важка операція обмежена латентністю арифметики. 3) Власний алгоритм дає ≈2× лише коли послідовне виконання повільне (-O0 або MulMod) і N ≥ 10^6; максимум прискорення обмежений двома проходами даних і пропускною здатністю пам’яті, а не кількістю потоків. 4) K слід брати порядку кількості потоків процесора (або менше), а великі K (сотні) шкідливі через лінійне зростання накладних витрат.